4 January 2026

SOFTWARE REQUIREMENTS SPECIFICATION

Keylogging Detection

version 1.3

Primary Advisor **Sir Rauf Butt**

PREPARED BY **MUHAMMAD MAMOON, NATASHA ZAHEER, HAIDER ALI**
261938489 261934680 261941967

Table of Contents

Table of Diagrams

Revision History

Name	**Date**	**Reason For Change**	**Version**
Haider Ali	26th December	Initial Draft	1.0
Muhammad Mamoon	2nd January	Minor changes	1.1
Natasha Zaheer	3rd January	Diagrams added	1.2
Muhammad Mamoon	3rd January	Migrated Independent Verification & Validation table from testing results	1.3

1. INTRODUCTION

1.1. PURPOSE

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements for the Linux Keylogger Detection System. This document provides a comprehensive description of the system's architecture, which operates across Kernel Space and User Space to detect malicious keyboard monitoring behavior.

The primary objective of the software is to implement a behavioral-based detection system that identifies keyloggers through timing anomalies and process context analysis, rather than relying on traditional signature-based methods. The system employs machine learning models for anomaly detection, supported by rule-based heuristic algorithms that provide additional detection signals and handle edge cases. This project serves to demonstrate kernel-level security monitoring techniques while adhering to strict privacy-by-design principles.

1.2. INTENDED AUDIENCE AND READING SUGGESTIONS

This document is intended for the following stakeholders involved in the academic evaluation and technical review of the project:

1. **Project Supervisors and Academic Examiners:**
    - **Focus:** Sections 1.4 (Scope) and the Ethical Considerations to understand the privacy-preserving nature of the research.
    - **Suggestion:** Review the "Research Contributions" to differentiate this behavioural approach from traditional antivirus solutions.
2. **Security Researchers and Developers:**
    - **Focus:** System Architecture, specifically the Kernel Module logic (`fyp_kbd.c`) and Netlink communication protocol.
    - **Suggestion:** Pay close attention to the "Detection Heuristics" (Rapid Typing, Burst Pattern, Unknown Process) detailed in the Daemon documentation.
3. **Linux System Administrators:**
    - **Focus:** Installation procedures, Resource Monitoring requirements, and Interpretation of Security Alerts.
    - **Suggestion:** Review the "False Positive Analysis" in the Testing section to understand operational limitations.

1.3. PROJECT SCOPE

The **Linux Keylogger Detection System** is a host-based security tool designed for the **Ubuntu 22.04 LTS** operating system running **Linux Kernel 5.15.x**.

1.3.1. In-Scope Functions

The system **shall** perform the following functions:

- **Kernel-Level Monitoring:** Load a kernel module (`fyp_kbd.ko`) to hook the `keyboard_notifier_list` and observe keyboard events at Ring 0.

- **Metadata Collection:** Collect behavioral metadata including process ID (PID), process name (comm), command line (cmdline), and inter-keystroke timing, **without** capturing keystroke content (keycodes).
- **Real-Time Communication:** Transmit event data from kernel space to user space via Netlink Protocol 31 (`NETLINK_FYP_DETECTOR`) with <1ms latency. Events are delivered as 158-byte structured messages containing timestamp, PID, process name, command line, event type, and rapid flag.
- **Behavioural Analysis:** A Python daemon (fyp_daemon.py) shall analyze events using machine learning-based anomaly detection, supported by three rule-based heuristic algorithms:
    1. **Rapid Typing Detection:** Triggers when >50% of a process's keyboard events occur within <50ms intervals (configurable via sysfs parameter `rapid_threshold_ms`).
    2. **Burst Pattern Detection:** Triggers when a process's event rate exceeds 100 events/second sustained over a 2-second window (configurable via `sysfs` parameter `burst_threshold_eps`)
    3. **Unknown Process Detection:** Identification of non-whitelisted processes accessing the keyboard input stream, where the whitelist includes known system processes (gnome-shell, Xorg, bash, terminals, etc.).
- **Visualization:** Provide a native Qt-based desktop GUI application (PySide6/Qt6) for real-time alerting, process statistics, event rate visualization with 60fps animated charts, and resource monitoring (combined CPU/Memory usage of GUI and daemon processes).

1.3.2. Out-of-Scope (Exclusions)

The following are explicitly **excluded** from the project scope:

- **Malware Removal:** The system is a *detector* only; it will attempt to terminate malicious processes but will not be allowed to delete files.
- **Signature-Based Detection:** The system will not scan file binaries or use hash databases to detect known malware.
- **Keystroke Logging:** The system will strictly **not** record, store, or transmit any actual keycodes, passwords, or typed content.
- **Cross-Platform Support:** The system is designed exclusively for the Linux kernel architecture and will not support Windows or macOS.
- **Hardware Keylogger Detection:** The system detects software-level hooks and file stream reading; it cannot detect physical hardware devices attached to the USB port.

2. OVERALL DESCRIPTION

This section provides a comprehensive overview of the system being developed, its operational context, intended users, and constraints. It establishes a conceptual understanding of the product before presenting detailed functional and non-functional requirements in subsequent sections. The system aims to detect keylogging behavior at the kernel level, analyze it using machine learning-based behavioral analysis supported by rule-based heuristics, and present meaningful, real-time insights to users through a native desktop dashboard application.

2.1. DOCUMENT CONVENTIONS

This Software Requirements Specification (SRS) document follows the general structure and terminology recommended by the IEEE 830/29148 standard for software requirements documentation.

The following conventions are used throughout this document:

- **"Shall"** indicates a mandatory requirement that the system must fulfill.
- **"Should"** indicates a recommended feature that enhances the system but is not strictly required.
- **"May"** indicates an optional feature that can be implemented if time and resources permit.
- Technical terms related to operating systems, kernel development, cybersecurity, and machine learning are written in *italics* upon first occurrence.
- Acronyms such as **LKM** (Loadable Kernel Module), **VM** (Virtual Machine), **ML** (Machine Learning), and **PID** (Process Identifier) are defined when first introduced.
- Diagrams referenced in the document (e.g., system architecture, data flow diagrams) are assumed to be included in later sections or appendices.

2.2. PRODUCT PERSPECTIVE

The proposed system is a **standalone Linux-based security monitoring solution** designed to provide visibility into keyboard input behaviour at the kernel level and to detect anomalous activity indicative of keyloggers.

The system adopts a **layered architecture** consisting of:

1. **Kernel Space Component**
   A custom Loadable Kernel Module that integrates with the Linux input subsystem to monitor keyboard events in real time.
2. **User Space Analysis Component**
   A background Python daemon (`fyp_daemon.py`) that receives behavioral telemetry from the kernel module via Netlink sockets, tracks per-process statistics, applies machine learning-based anomaly detection supported by rule-based heuristics, and determines whether observed behavior is benign or suspicious.
3. **Presentation and Awareness Layer**
   A native Qt-based desktop application (built with PySide6) that visualizes live system activity through real-time charts, highlights detected anomalies with severity-coded alerts, monitors system resource usage, and provides an intuitive interface for

security awareness. The GUI reads daemon status via a JSON file-based IPC mechanism (`/tmp/fyp_status.json`) updated every 500ms.

The product does not aim to replace traditional antivirus or endpoint protection software. Instead, it complements existing solutions by focusing on behavioural detection, kernel-level visibility, and user awareness, areas where signature-based tools often fall short.

2.3. PRODUCT FEATURES

The system provides the following major features:

2.3.1. Kernel-Level Keyboard Monitoring

- The system monitors keyboard input events directly within the Linux kernel using a Loadable Kernel Module.
- Events are captured before they reach user-space applications, making the system resilient to user-space evasion techniques.

2.3.2. Real-Time Telemetry Collection

- Keyboard event metadata such as timestamps, event frequency, and associated process context are collected continuously.
- Data is transmitted from kernel space to user space using a secure and efficient communication mechanism.

2.3.3. Behavioral Feature Extraction

- The system extracts behavioral features including: inter-keystroke timing intervals (flagging events occurring within the configurable `rapid_threshold_ms`), per-process event counts and rates (events per second), rapid event ratio (percentage of events below timing threshold), process context (PID, process name, full command line), and event type (press/release).
- These features form the basis for machine learning-based anomaly detection, with rule-based heuristics providing additional detection signals.

2.3.4. Machine Learning-Based Detection with Heuristic Support

- The system employs lightweight machine learning models for unsupervised or semi-supervised anomaly detection, suitable for identifying unknown or zero-day threats based on learned behavioral patterns.
- Three rule-based heuristic detection algorithms (Rapid Typing Detection, Burst Pattern Detection, and Unknown Process Detection) provide supporting detection signals and handle specific edge cases where ML models may require additional context.

2.3.5. Background Execution

- The detection mechanism runs continuously as a background service without requiring user interaction.
- The system is designed to have minimal impact on system performance.

2.3.6. Live Monitoring Dashboard

- A native Qt-based desktop dashboard (PySide6) provides real-time visualization of system activity with 60fps animated charts.

- Users can observe keyboard event rate trends, per-process statistics, detection alerts with severity levels, and combined resource usage (CPU/Memory) of the detection system as they occur.

2.3.7. User Awareness and Educational Messaging

- In the event of suspicious activity, the system presents friendly and non-alarming notifications.
- Educational content explains what keyloggers are, why the activity is concerning, and how users can reduce their risk.

2.4. USER CLASSES AND CHARACTERISTICS

2.4.1. General Users

- Limited technical knowledge of operating systems.
- Interested in privacy and security awareness.
- Interact primarily with the dashboard to view alerts and system activity.
- Do not directly configure kernel-level components.

2.4.2. Advanced Users / System Administrators

- Familiar with Linux systems and administrative tasks.
- Capable of installing kernel modules and managing system services.
- May adjust configuration parameters such as detection thresholds or logging verbosity.

2.4.3. Developers / Researchers

- Strong understanding of operating systems, cybersecurity, and software engineering concepts.
- Use the system for experimentation, research, and analysis.
- Interested in the internal working of the kernel module, data pipeline, and detection logic.

2.5. OPERATING ENVIRONEMNT

The system operates within the following environment:

- **Operating System:** Linux (single, fixed kernel version)
- **Kernel Configuration:** Supports Loadable Kernel Modules
- **Execution Platform:** Linux Kernel 5.15.x
- **Programming Languages:** C (kernel module), Python and/or C (user-space daemon)
- **Dashboard Technologies:** PySide6 (Qt6 for Python), QtCharts for real-time visualization, psutil for resource monitoring. The application uses a GitHub-inspired dark theme with responsive layout design
- **Privileges:** Root access required for kernel module installation and service initialization

2.6. DESIGN AND IMPLEMENTATION CONSTRAINTS

The system is subject to several design and implementation constraints:

- The system is restricted to a **specific Linux kernel version**, limiting portability.

- Kernel module development introduces risks such as system instability and requires careful memory and concurrency management.
- Secure Boot must be disabled in the testing environment to allow kernel module loading.
- Machine learning models must be computationally efficient to avoid excessive CPU or memory usage.
- Development time and resources are limited due to academic project timelines.

2.7. USER DOCUMENTATION

The following documentation will be provided with the system:

- **Installation Manual:** Instructions for setting up the application, installing dependencies, and loading the kernel module.
- **User Guide:** Explanation of dashboard features, visualizations, and alert messages.
- **Security Awareness Guide:** Educational material on keyloggers, malware risks, and safe computing practices.
- **Developer Documentation:** High-level explanation of system architecture, component interactions, and data flow.

2.8. ASSUMPTIONS AND DEPENDENCIES

The system is **developed** under the following assumptions and dependencies:

- The target system uses the supported Linux kernel version.
- Users have sufficient privileges to install kernel modules.
- Required kernel headers and build tools are available.
- VirtualBox correctly forwards keyboard input events to the guest operating system.
- Training data used for machine learning accurately represents normal and malicious behavior.
- No network connectivity is required; the system operates entirely locally using file-based IPC (`/tmp/fyp_status.json`) between the daemon and GUI.

3. EXTERNAL INTERFACE REQUIREMENTS

This section describes all external interfaces through which the system interacts with users, hardware, software components, and communication mechanisms. These interfaces define how data flows into, within, and out of the system, ensuring interoperability, usability, and extensibility.

3.1. USER INTERFACES

The primary user interface for the system is a **native Qt-based desktop application** (built with PySide6/Qt6) designed to provide real-time visibility into system behavior and detection results. The application features a modern GitHub-inspired dark theme (#0d1117 background) with responsive 2-column layout.

3.1.1. Dashboard Interface

The dashboard shall provide the following interface elements across seven navigation pages (Dashboard, Alerts, Processes, Event Stream, AI Assistant placeholder, ML Insights placeholder, Configuration):

- **Live Activity View**
  - Displays real-time keyboard activity statistics derived from kernel-level monitoring.
  - Visualizes behavioural trends rather than raw keystrokes, preserving user privacy.
- **Detection and Alert Panel**
  - Shows the current detection state through severity-coded indicators (LOW/MEDIUM/HIGH).
  - Displays alerts generated by the machine learning detection engine and supporting heuristic rules, including anomaly scores and confidence levels.
  - Presents contextual information explaining detected anomalies, including the triggering detection mechanism, process details, and specific threshold violations.
- **Process Monitoring View**
  - Presents contextual information explaining detected anomalies, including the triggering detection mechanism, process details, and specific threshold violations.
  - Highlights processes flagged by the ML detection engine and supporting heuristics as suspicious, color-coded by severity.
- **User Awareness and Education Section**
  - Provides explanatory messages about keyloggers and behavioural monitoring.
  - Displays guidance on preventive security practices in a non-alarming manner.
- **System Status Indicators**
  - Shows the operational status of the kernel module (via procfs presence check) and user-space daemon (via status file monitoring.

- Displays real-time resource usage (combined CPU percentage and memory MB) for both GUI and daemon processes via `psutil`.
- Provides 60-second history charts for resource trends with color-coded thresholds (blue: normal, yellow: elevated, red: high).

The user interface is intended to be **informational and educational**, avoiding intrusive prompts or disruptive alerts.

3.1.2. Interaction Characteristics

- The dashboard is **read-only** for general users.
- Administrative configuration is intentionally minimal to reduce misuse or misconfiguration.
- No direct interaction with kernel-space components is exposed to end users.

3.2. HARDWARE INTERFACES

The system interacts indirectly with hardware through the Linux kernel and does not communicate with physical devices directly.

3.2.1. Input Devices

- Standard keyboard input devices connected to the system.
- Input events are captured via the Linux input subsystem within the kernel.

3.2.2. Execution Hardware

- The system is developed and tested on **x86-based systems running Linux kernel 5.15.x**. (Virtualized in development**)**
- Virtualization is provided by **Oracle VirtualBox**, which forwards hardware input events to the guest operating system.

No specialized hardware or external peripherals are required for system operation.

3.3. SOFTWARE INTERFACES

The system relies on well-defined software interfaces across kernel space, user space, and the presentation layer.

3.3.1. Kernel–User Space Interface

- The Loadable Kernel Module exports telemetry data to user space using Netlink Protocol 31 (`NETLINK_FYP_DETECTOR`).
- Data is transmitted as 158-byte structured events (struct `fyp_netlink_event`) containing: 64-bit nanosecond timestamp, 32-bit PID, 16-byte process name, 128-byte command line, event type (press/release), and rapid flag.
- The daemon registers its PID with the kernel module for unicast event delivery with <1ms latency.
- Raw keystroke values (keycodes) are explicitly NOT captured or transmitted, maintaining privacy by design

This interface serves as the **foundation of the system's novelty**, enabling kernel-level behavioral data collection for machine learning-based analysis with heuristic support.

3.3.2. Detection Engine Interface

- The system uses a Detection Engine that combines machine learning-based anomaly detection with three supporting heuristic rules (Rapid Typing, Unknown Process, Burst Pattern) using configurable thresholds.
- The daemon tracks per-process statistics (ProcessStats dataclass) including total events, rapid events, rapid ratio, and events per second.
- The machine learning model consumes behavioral feature vectors for learned pattern-based detection, while heuristics provide additional signals for specific anomaly types.

3.3.3. Dashboard Backend Interface

- The dashboard communicates with the daemon via file-based IPC:
  - The daemon writes status to `/tmp/fyp_status.json` **every 500ms.**
  - The GUI polls this file at 500ms intervals to retrieve live data.
  - Configuration changes (e.g., burst threshold) are written to `/tmp/fyp_daemon_config.json` and picked up by the daemon.
- Data is structured as JSON containing: timestamp, daemon status, kernel module status, total event count, per-process statistics, and alert list.

3.3.4. Operating System Interfaces

The system interfaces with the Linux operating system for:

- Kernel module loading and unloading
- Background service management
- Process identification and context retrieval
- File system access for logging and model storage

3.4. COMMUNICATIONS INTERFACES

The system uses internal and local communication interfaces to enable real-time data flow between components.

3.4.1. Kernel to User-Space Communication

- A low-latency communication channel is used to stream behavioural telemetry from the kernel module to the user-space daemon.
- Communication is local to the system and does not involve external networks.

3.4.2. Inter-Process Communication (IPC)

- The user-space daemon communicates internally with the machine learning engine.
- IPC mechanisms are optimized for low overhead and real-time processing.

3.4.3. Dashboard Communication

- The dashboard (GUI) communicates with the daemon via local file-based IPC, specifically through JSON files in `/tmp/`.
- Real-time updates are achieved through 500ms polling of the status file, with smooth 60fps chart animations for visual continuity.
- No network connectivity (local or external) is required; the entire system operates through kernel-userspace Netlink sockets and filesystem-based IPC.

3.4.4. Security Considerations

- All communication interfaces are restricted to the local system.
- No sensitive input data or keystroke content is transmitted outside the system.
- The design minimizes the attack surface by limiting exposed interfaces.

4. REQUIREMENTS

4.1. FUNCTIONAL REQUIREMENTS

4.1.1. Kernel-Level Monitoring

FR-1
The system shall monitor keyboard input events at the kernel level using a Loadable Kernel Module (LKM).
FR-2
The kernel module shall operate transparently without altering normal keyboard functionality.
FR-3
The kernel module shall collect behavioral metadata associated with keyboard input events, including timing and process context.
FR-4
The system shall avoid logging or storing raw keystroke content to preserve user privacy.

4.1.2. Kernel to User-Space Data Transfer

FR-5
The kernel module shall transmit collected behavioral telemetry to a user-space component in real time.
FR-6
The data transfer mechanism shall be efficient and minimize performance overhead.
FR-7
The communication interface shall restrict access to authorized system components only.

4.1.3. User-Space Data Processing

FR-8
The system shall execute a background user-space daemon to process telemetry received from the kernel module.
FR-9
The daemon shall perform feature extraction on kernel telemetry to generate behavioral feature vectors.
FR-10
Extracted features shall represent user and process behavior rather than individual keystrokes.

4.1.4. Machine Learning-Based Detection

FR-11
The system shall employ machine learning models to perform behavioral anomaly detection, identifying patterns that deviate from learned normal behavior.
FR-12
The detection engine shall analyze per-process behavioral statistics including event counts, rapid event ratios, events per second, and generate anomaly scores.
FR-13
The system shall classify observed behavior as normal or potentially suspicious based on ML-generated anomaly scores and confidence levels.
FR-14
Rule-based heuristic algorithms (Rapid Typing Detection, Burst Pattern Detection,

Unknown Process Detection) shall provide supporting detection signals and handle edge cases where ML models benefit from additional context.

4.1.5. Alerting and Awareness

FR-15
The system shall generate an alert when the detection engine identifies anomalous behavior exceeding defined thresholds. Alerts shall include ML-generated anomaly scores and be classified by severity: HIGH (ML high-confidence anomaly, Rapid Typing, Burst Pattern), MEDIUM (ML medium-confidence anomaly, Unknown Process).
FR-16
Alerts shall be presented in a non-intrusive and user-friendly manner, with color-coded severity indicators and clear process identification.
FR-17
The system shall provide contextual information explaining the reason for detected anomalies, including anomaly scores, confidence levels, triggering mechanisms, threshold values, and measured metrics.

4.1.6. Dashboard Functionality

FR-18
The system shall provide a native Qt-based desktop dashboard (PySide6) for live monitoring, with a responsive 2-column layout and GitHub-inspired dark theme.
FR-19
The dashboard shall display real-time behavioral metrics (event rates, per-process statistics) and detection results (alerts with severity levels) updated every 500ms.
FR-20
The dashboard shall visualize event rate trends over time using animated spline charts (60fps), process activity via bar charts, and resource usage history via line charts
FR-21
The dashboard shall indicate the operational status of system components (kernel module loaded status via procfs, daemon running status via status file), and display combined CPU/Memory usage of the detection system.

4.1.7. System Management

FR-22
The system shall allow authorized users to start and stop the detection service.
FR-23
The system shall support loading and unloading of the kernel module during runtime.
FR-24
The system shall log detection events for auditing and analysis purposes.

4.2. NON-FUNCTIONAL REQUIREMENTS

4.2.1. Performance Requirements

NFR-1
The system shall operate with minimal impact on system performance.
NFR-2
Kernel-level monitoring shall not introduce noticeable latency in keyboard input handling.
NFR-3
The user-space daemon shall process incoming telemetry in near real time.

4.2.2. Reliability and Stability

NFR-4
The system shall operate continuously without requiring frequent restarts.
NFR-5
Failure of the dashboard or user-space components shall not compromise kernel stability.
NFR-6
The kernel module shall handle unexpected conditions gracefully to avoid system crashes.

4.2.3. Security Requirements

NFR-7
The system shall restrict kernel module interaction to privileged users.
NFR-8
Communication between system components shall be limited to local interfaces.
NFR-9
The system shall not transmit captured data outside the host machine.

4.2.4. Privacy Requirements

NFR-10
The system shall not record or store raw keystrokes or sensitive user input.
NFR-11
Only behavioral and statistical metadata shall be used for analysis.

4.2.5. Usability Requirements

NFR-12
The dashboard interface shall be intuitive and understandable to non-technical users.
NFR-13
System alerts shall use clear and non-alarming language.

4.2.6. Scalability and Extensibility

NFR-14
The system design shall support future enhancement of machine learning models.
NFR-15
Additional behavioural features may be incorporated without redesigning the kernel module.

4.2.7. Maintainability

NFR-16
System components shall be modular and independently maintainable.
NFR-17
The system shall include sufficient logging to aid debugging and analysis.

4.2.8. Portability Constraints

NFR-18
The system shall operate on a specific Linux kernel version defined at development time.
NFR-19
Portability to other kernel versions is not a primary requirement.

5. SYSTEM ANALYSIS AND DESIGN

5.1. ACTIVITY DIAGRAM

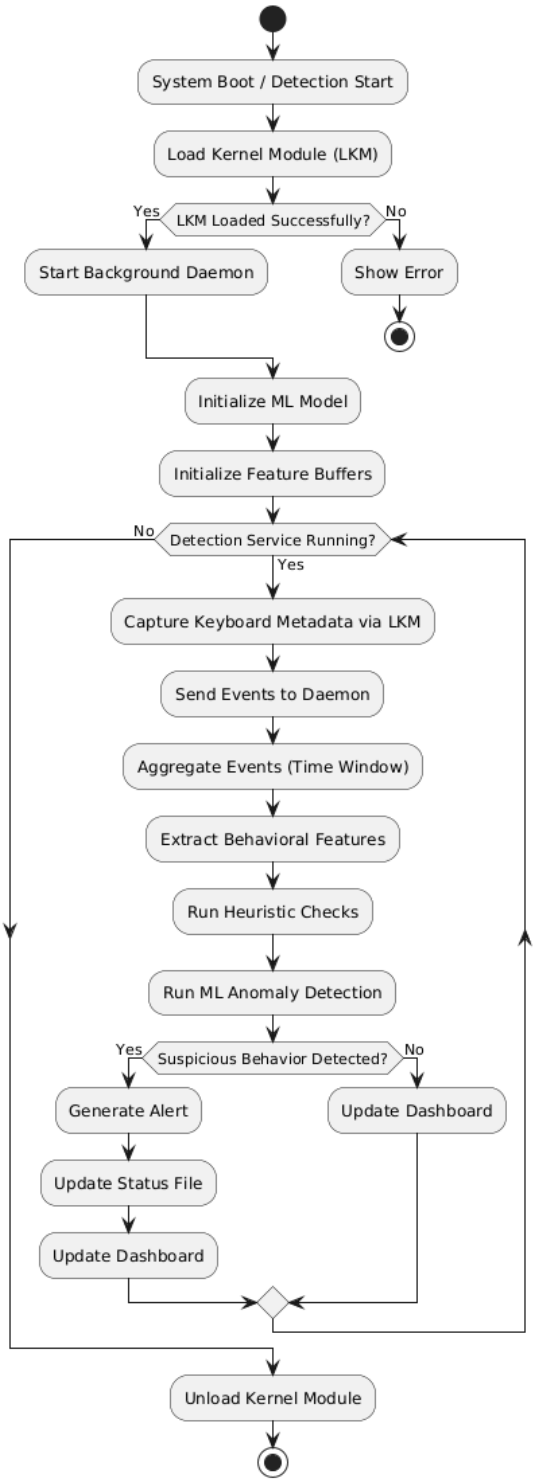

Diagram 1: Activity Diagram

5.2. USE CASE DIAGRAM

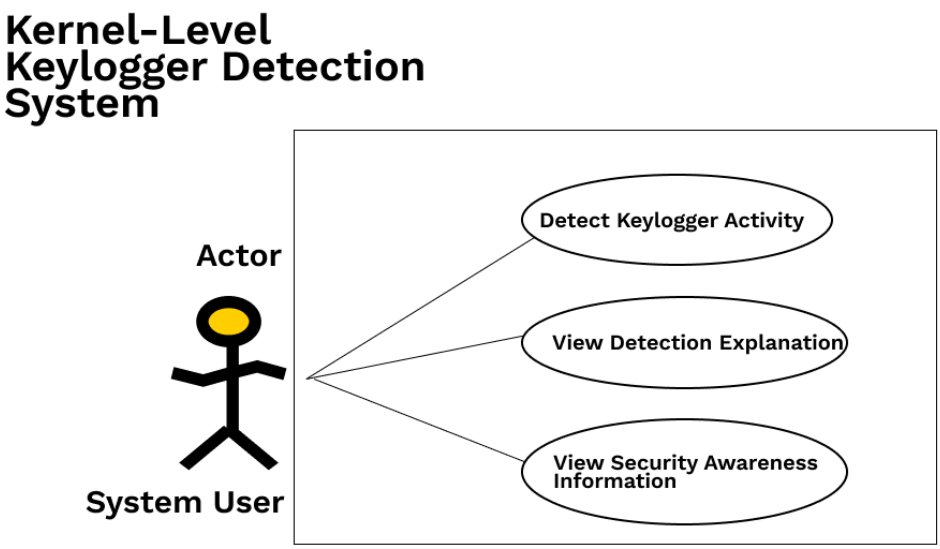

Diagram 2: Use Case Diagram

5.3. SEQUENCE DIAGRAM

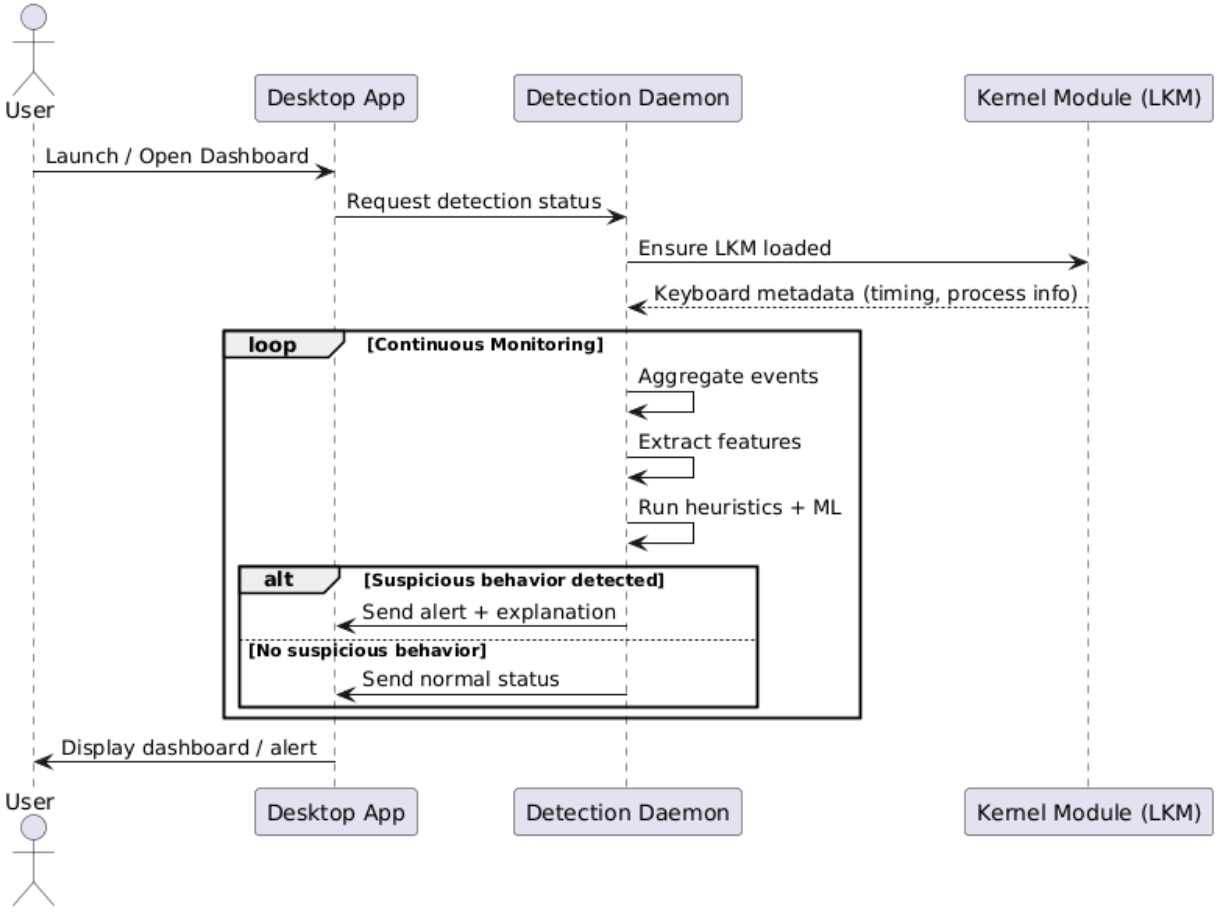

Diagram 3: Sequence Diagram

5.4. STATE DIAGRAM

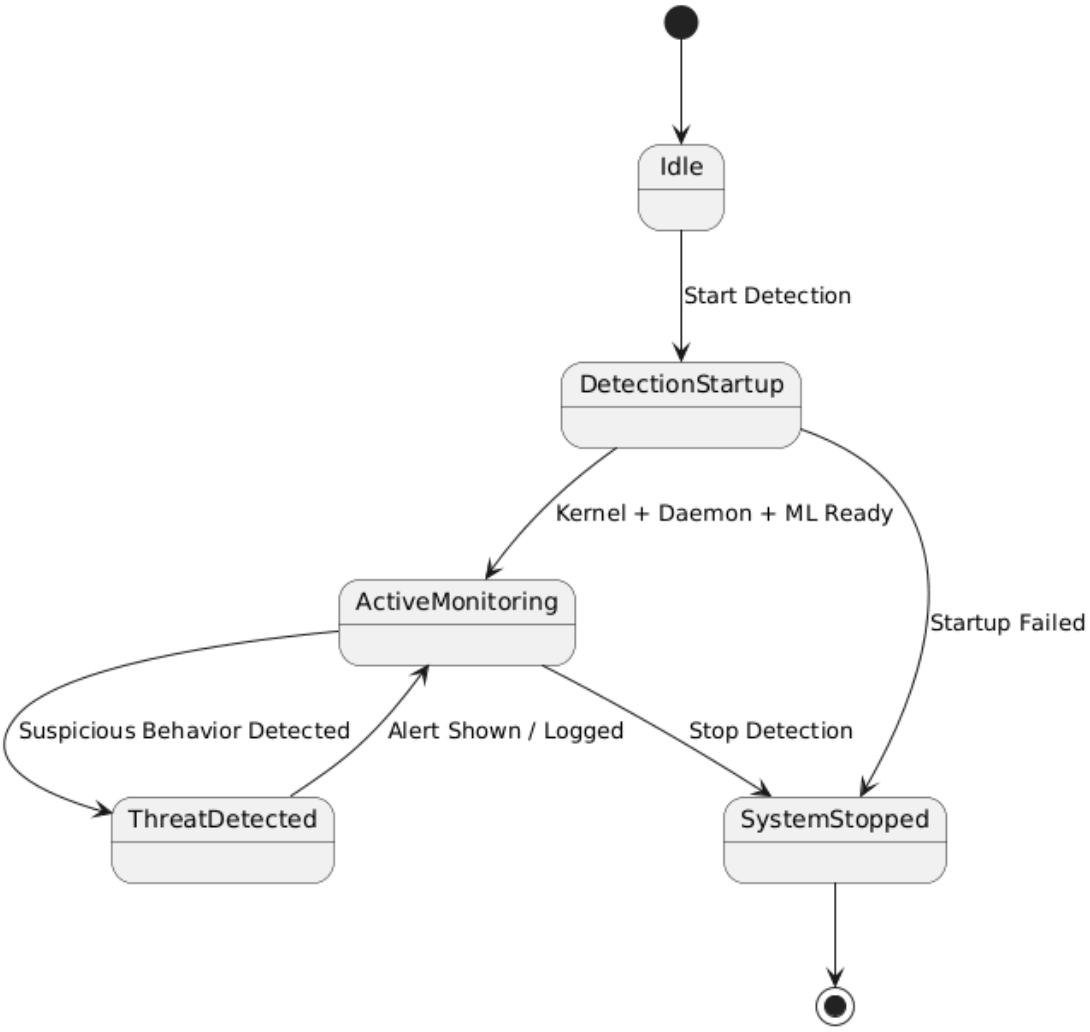

Diagram 4: State Diagram

5.5. ARCHITECTURE DIAGRAM

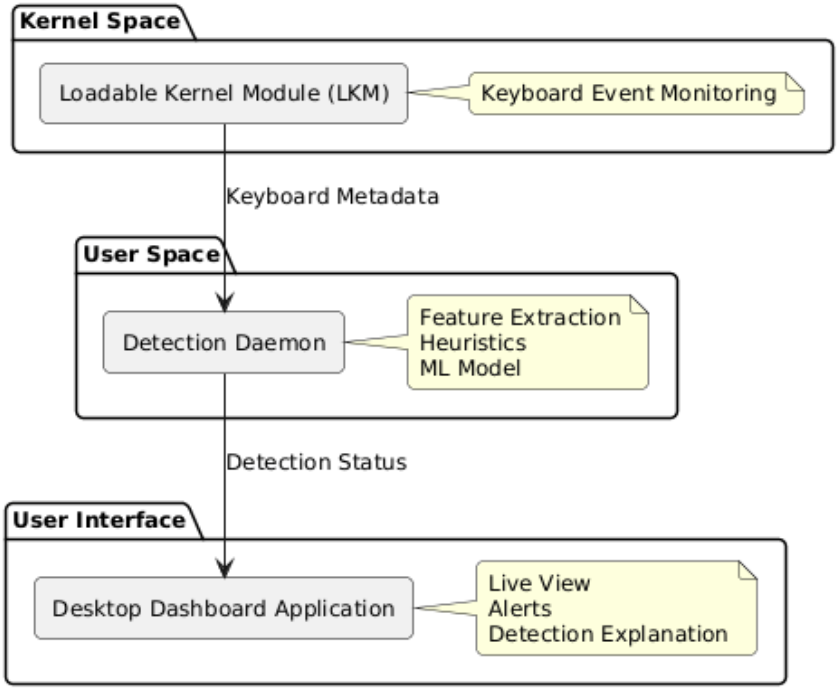

Diagram 5: Architecture Diagram

6. REFERENCES

1. *Chen, J., & Wang, H. (2023). AI-based detection of keylogging attacks using system call traces. IEEE Access, 11, 10532–10547.*
2. *CrowdStrike. (2024). Using artificial intelligence for real-time threat hunting and detection (Whitepaper). Retrieved from https://www.crowdstrike.com/resources*
3. *Microsoft. (2023). AI-driven threat detection in endpoint security (Microsoft Security Blog). Retrieved from https://www.microsoft.com/security*
4. *Sahoo, S., Pradhan, R., & Mishra, D. P. (2024). A deep learning framework for detecting malicious keyloggers in Linux environments. Journal of Cybersecurity, 10(1).*
5. *SentinelOne. (2023). Behavioral AI for endpoint detection and response. SentinelLabs Technical Report. Retrieved from https://www.sentinelone.com/labs*
6. *Zhang, T., & Wang, Y. (2022). Kernel-level anomaly detection using machine learning. Computers & Security, 114, 102594. (Journal)*
7. *Aqua Security. (2024). Tracee — runtime security and forensics using eBPF. Retrieved from https://aquasecurity.github.io/tracee/ (Tracee is a practical eBPF-based runtime security tool widely used for kernel-level telemetry). aquasecurity.github.io*
8. *Aqua Security. (2022). Detecting and capturing kernel modules with Tracee and eBPF. Aqua Security Blog. Retrieved from https://www.aquasec.com/blog/linux-security-with-tracee-and-ebpf/. Aqua*
9. *K. Huang et al. (2025). SoK: Challenges and paths toward memory safety for eBPF. (analysis of risks and verification challenges in the eBPF ecosystem). Retrieved from https://nebelwelt.net/files/25Oakland.pdf. nebelwelt.net*
10. *Yu, Y. C. (2025). Kernel-level hidden rootkit detection based on eBPF. Information & Computer Security (article)*

Appendix A: Glossary

- **LKM (Loadable Kernel Module)**
  A dynamically loadable component that extends the Linux kernel without requiring a system reboot.
- **Kernel Space**
  A privileged execution environment in which the operating system kernel operates and has full access to hardware resources.
- **User Space**
  A non-privileged execution environment where user applications and background services run.
- **PID (Process Identifier)**
  A unique numeric identifier assigned by the operating system to each running process.
- **IPC (Inter-Process Communication)**
  Mechanisms that allow different software components to exchange data locally on the same system.
- **Machine Learning (ML)**
  A technique that enables systems to learn patterns from data and identify anomalies without explicit rule definitions.
- **Heuristic Detection**
  Rule-based detection methods that identify suspicious behavior using predefined thresholds and conditions.
- **Anomaly Detection**
  The process of identifying behavior that deviates significantly from normal system activity.
- **Netlink**
  A Linux kernel–to–user-space communication mechanism used for transferring structured data efficiently.

Appendix B: IV & V Report

Independent verification & validation report

Sr #	Defect	Origin Stage	Status
1	**Kernel panic on cmdline access in atomic context –** Keyboard notifier runs in atomic context but original code attempted to access mm_struct which requires sleeping. Caused kernel crashes.	**Prototype Implementation**	**Fixed**
2	**Netlink message size mismatch –** Daemon expected 30-byte events but kernel sent 158-byte events after adding cmdline field. Caused struct unpacking errors.	**Prototype Implementation (Netlink integration into LKM)**	**Fixed**
3	**GUI freezing during status file read –** Synchronous file I/O on main thread blocked UI updates. Moved to polling with non-blocking reads.	**GUI Testing**	**Fixed**
4	**Daemon crash when kernel module not loaded –** socket.bind() failed with unclear error when module absent. Added checks to make sure LKM is loaded.	**Daemon + LKM Testing**	**Fixed**

5	**Procfs polling causing high CPU usage –** Daemon polled procfs every 10ms causing 15-20% CPU usage.	**Kernel Design**	**Fixed** - Migrated to event-driven Netlink sockets with <1 ms latency + more control instead of procfs.